

REINFORCEMENT LEARNING & ADVANCED DEEP

M2 DATA SCIENCE

TP DQN

Ce TP a pour objectif d'implémenter et expérimenter l'algorithme DQN vu en cours, sur différents environnements.

Télécharger et extraire l'archive env.zip sur le moodle, ainsi que le fichier requirements.txt et memory.py à mettre dans le même repertoire env.

Pour ce TP, nous allons utiliser le gestionnaire de librairies uv. Lancer dans un repertoire de téléchargement:

```
wget -q0- https://astral.sh/uv/install.sh | sh
uv venv rl --python 3.11.5
```

Se placer dans le repertoire du projet téléchargé. Puis:

```
source rl/bin/activate
pip install swig
uv pip install -r requirements.txt
uv pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

En cas de problèmes graphiques avec l'affichage des environnements, la commande suivante permet de régler le problème sur de nombreuses machines:

```
export LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libstdc++.so.6
```

Si problème persistant pour l'affichage de GridWorld: commenter la ligne:

```
matplotlib.use("TkAgg")
```

1 Plateforme Gym

Au cours de ce TP nous considérerons trois environnements différents, qui suivent tous le format de la plateforme de RL `gym` (de *open-ai*). Cette plateforme travaille avec quelques fonctions de base, utilisées de la même manière sur tous les environnements:

- initialiser un environnement (ici grid-world) : `env = gym.make('gridworld-v0')` pour créer l'env;
- Démarrage d'une nouvelle trajectoire en se plaçant sur un état de départ : `env.reset()` qui retourne l'état de départ de l'agent;

- `env.action_space` permet de connaître l'ensemble des actions possibles;
- `obs, reward, done, info = env.step(action)` permet de jouer l'action passée en argument : `obs` contient le nouvel état, `reward` la récompense associée à l'action, `done` un booléen pour savoir si le jeu est fini;
- `env.render()` permet de visualiser le jeu (si `env.verbose` est à `true`);

2 Environnement de travail

L'implémentation doit être réalisée en PyTorch. Dans l'archive extraire vous trouverez:

- `randomAgent.py` : une implémentation d'un agent aléatoire qui vous servira de point de départ à toutes vos implémentations. On donne la boucle principale de l'entraînement, après avoir chargé un fichier de configuration (fourni dans `configs`, voir plus bas). Il s'agira principalement dans ce TP de remplacer les fonctions `act` (qui choisit pour le moment une action de manière uniforme), `learn` (qui ne fait pour le moment rien mais dont il faudra donner le code pour faire évoluer l'agent en fonction de ses expériences) et `store` (qui devra stocker chaque transition d'entraînement dans un buffer);
- `core.py` : un fichier contenant diverses fonctions dont vous pourrez vous servir pour l'extraction de features à partir des observations et la construction de réseaux de neurones via `pytorch` (vous pourrez par exemple utiliser la classe `NN` donnée dans ce fichier pour implémenter la fonction Q du DQN).
- `utils.py`: fournit différentes fonctions utiles pour l'initialisation des environnements, le chargement des configs et la création des répertoires de résultats. La fonction appelée au démarrage de `randomAgent` lance notamment une instance de `tensorboard`, pratique pour visualiser des courbes d'apprentissage et test de votre agent, à l'adresse `localhost` donnée en début de sortie du programme;
- repertoire configs : un repertoire contenant des fichiers de configuration `yaml`. Ceux contenus actuellement correspondent aux fichiers de configuration utiles pour lancer `randomAgent` sur les 3 environnements que l'on considère dans ce TP. On donne aussi 3 fichiers de configs pour `dqn` que j'ai utilisés pour faire mes expériences de préparation du TP, dont vous pouvez vous inspirer;
- `memory.py` : déclare une structure de données `Memory` pratique pour constituer le replay buffer. Cette structure permet de stocker les transitions (via la fonction `store`), d'échantillonner des transitions (via la fonction `sample`) et de modifier leur valeur de priorité (via la fonction `update`) de manière efficace. Si l'objet est initialisé avec la valeur `prior=True`, alors les transitions sont échantillonnées en fonction de leur écart de différence temporelle, accompagnées de leur poids d'Importance Sampling utile pour débiaiser l'algorithme DQN (commencer à travailler avec cette variable à `False`, puis l'activer ensuite pour essayer d'améliorer les performances).

3 Environnements de jeu

Dans un premier temps, vous testerez `randomAgent` sur chaque fichier de configuration fourni dans le repertoire configs:

- `config_random_gridworld.yaml` : configuration pour l'environnement `gridWorld` discuté en cours. L'environnement `gridworld-v0` est une tâche de RL où un agent (point bleu) doit récolter des éléments jaunes dans un labyrinthe 2D et terminer sur une case verte, tout en évitant les cases roses (non terminales) et rouges (terminales). La configuration des rewards utilisée est donnée dans le fichier de config, ainsi que la carte à charger pour cette environnement (10 cartes dispos).
- `config_random_cartpole.yaml` : configuration pour l'environnement `Cartpole` de gym. `Cartpole` est un jeu où l'on cherche à stabiliser une barre verticale sur un chariot roulant, en fonction de ses mouvements. Deux actions possibles à chaque itération: gauche ou droite. Le jeu est perdu si la barre verticale s'incline de plus de 15 % (elle tombe alors inévitablement) ou si le chariot sort de l'écran. Le reward cumulé correspond au nombre de pas de temps sans terminaison du jeu (maximum 500 pas de temps). Les observations sont des vecteurs de 4 réels donnant la position du chariot, sa vitesse, l'inclinaison de la barre et sa vitesse de rotation. C'est a priori l'environnement le plus simple.
- `config_random_lunar.yaml` : configuration pour l'environnement `LunarLander` de gym. `LunarLander` est un jeu où l'objectif est de faire alunir une fusée, qui a une vitesse et une direction initiales aléatoires. Le reward obtenu dépend de la vitesse à laquelle la fusée s'est posée et de la distance de la cible (ainsi que du temps mis à alunir, du fait que les deux pieds touchent le sol, du fuel dépensé, etc.). 4 actions possibles par pas de temps: ne rien faire, allumer le moteur de gauche, le moteur central ou le moteur de droite. Les observations sont des vecteurs de 8 réels donnant diverses informations sur la position, la direction et la vitesse de la fusée.

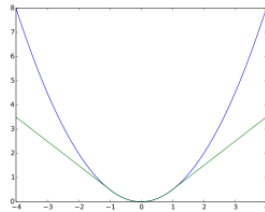
Vous noterez que la classe `randomAgent` fournie instancie une variable `featureExtractor`, pas utile pour `randomAgent`, mais dont vous aurez besoin pour extraire une représentation des observations fournies par l'environnement, en utilisant la fonction `getFeatures` de la classe correspondante. Selon le type d'extracteur souhaité, cette variable est définie différemment dans le fichier de configuration. Pour `Cartpole` et `LunarLander`, c'est une fonction qui ne touche pas au vecteur fourni par l'environnement. Pour `GridWorld`, le fichier de configuration déclare un transformateur dont la fonction `getFeatures` prend en argument l'observation de l'environnement et retourne un vecteur de features contenant la position de l'agent, celles des éléments jaunes ainsi que celles des éléments roses (les éléments rouges et verts étant fixes, pas besoin de les représenter dans le vecteur de caractéristiques).

4 Travail à réaliser

Il s'agit de faire évoluer `randomAgent.py` pour en faire un DQN.

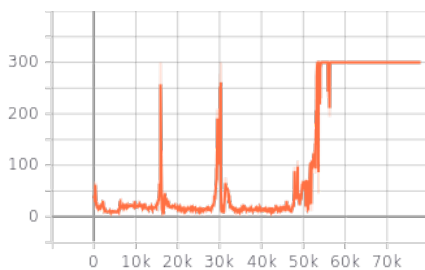
⇒ Produire des courbes d'apprentissage pour les 3 jeux, en comparant des versions avec Experience Replay (Prioritized ou non) et Target Network et des versions sans, selon différents jeux d'hyper-paramètres.

À noter que pour l'apprentissage de la fonction Q, il est souvent préférable d'utiliser un coût Huber (`torch.nn.SmoothL1Loss`) plutôt qu'un coût moindre carré classique, afin d'éviter l'explosion des gradients:

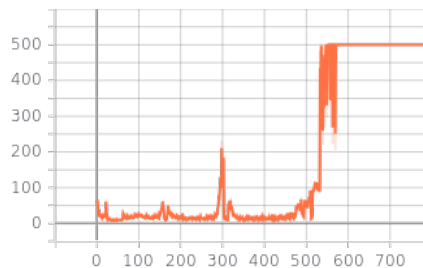


$$L_{\delta}(a) = \begin{cases} \frac{1}{2}a^2 & \text{for } |a| \leq \delta, \\ \delta(|a| - \frac{1}{2}\delta), & \text{otherwise.} \end{cases}$$

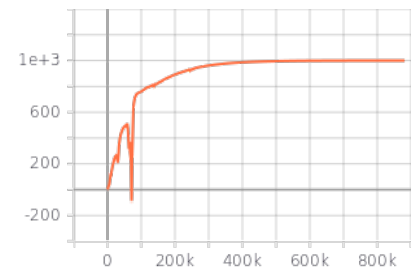
À titre indicatif, voici ce que l'on peut obtenir sur Cartpole, avec DQN utilisant prioritized replay et un target network (mis à jour tous les 1000 événements), une exploration epsilon-greedy (epsilon initial = 0.1, avec decay de 0.99999 à chaque événement, un discount de 0.999, un pas d'apprentissage de 0.0003, un batch de 100 transitions, un replay buffer de 10000 transitions, un pas d'optimisation tous les 10 événements, une taille d'épisode maximale de 300 événements en apprentissage (500 en test) et selon un réseau de neurones à une couche cachée de 200 neurones (avec activation tanh sur la couche cachée):



(a) Reward en apprentissage



(b) Reward en test (1 test tous les 100 épisodes d'apprentissage)



(c) Valeur Q moyenne (abscisse = nombre d'évènements)

Bonus

- Implémenter la version Double DQN et comparer les résultats
- Implémenter la version Dueling-DQN et comparer les résultats
- Implémenter la version Noisy-DQN et comparer les résultats